1.insertion at beginning

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning()
{
    struct node *first, *second, *last;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    last = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    last->data = 30;
```

Output

```
After inserting: 1 10 20 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
    first->next = second;
    second->prev = first;

    second->next = last;
    last->prev = second;

    last->next = head;
    head->prev = last;
}

void insertStart(int data)
{
    struct node *newnode, *last;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    last = head->prev;

    newnode->next = head;
    newnode->prev = last;
```

Output

Clear

```
After inserting: 1 10 20 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    beginning();

    insertStart(1);

    printf("After inserting: ");
    display();

    return 0;
}
```

Output

```
After inserting: 1 10 20 30

=== Code Execution Successful ===
```

2. Deletion at End

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void end()
{
    struct node *first, *last;

    head = (struct node*)malloc(sizeof(struct node));
    first = (struct node*)malloc(sizeof(struct node));
    last = (struct node*)malloc(sizeof(struct node));

    head->data = 0;
    first->data = 1;
    last->data = 2;
```

Output

```
Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===
```

main.c

Share Run

```c

    head->prev = NULL;
    head->next = first;

    first->prev = head;
    first->next = last;

    last->prev = first;
    last->next = NULL;
}

void deleteEnd()
{
    struct node *delNode = head;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    while(delNode->next != NULL)
    {
```

Output

```
Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===
```

main.c

Share Run

```c
    }

    while(delNode->next != NULL)
    {
        delNode = delNode->next;
    }

    if(delNode->prev != NULL)
    {
        delNode->prev->next = NULL;
    }
    else
    {
        head = NULL;
    }

    free(delNode);
}

void display()
{
    struct node *temp = head;
```

Output

```
Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===
```

main.c

Share Run

```c
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    end();

    printf("Before deleting: ");
    display();

    deleteEnd();

    printf("\nAfter deleting: ");
    display();

    return 0;
}
```

Output

```
Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===
```